

RAILWAY MAINTENANCE PLANNING

RailSync AI

SIH 2026 · PS26027

Solution Design • Software Requirements Specification • Implementation Contract

Version 1.0 | 15 September 2026 | Engineering handoff baseline

Prepared for the RailSync project team, mentors, evaluators and Codex implementers.

Design principle: ML estimates priority or risk. Optimization selects a schedule. Deterministic validation checks feasibility. A controller retains approval authority.

This is a proposed prototype specification, not a description of an already implemented or railway-certified system. All numerical examples are synthetic design fixtures unless explicitly stated otherwise. No performance improvement has been measured for RailSync in this package.

Contents

1. Executive summary	3
2. Problem interpretation and source status	4
3. Assumptions, boundaries and domain decisions	5
4. Research-informed design rationale	6
5. AI, ML, optimization and validation	7
6. Final logical architecture	8
7. Data flow and module contracts	9
8. Maintenance request system and ingestion	10
9. Lifecycle, actors and state transitions	11
10. Network, assets and operating restrictions	12
11. Timetable, occupancy and goods forecasts	13
12. COA and Corridor Availability Engine	14
13. Integrated Railway Planning State	15
14. Baseline planner specification	16
15. Priority and risk engine	17
16. Maintenance Opportunity Engine	18
17. Cross-department bundling and resources	19
18. CP-SAT mathematical model	20
19. Objectives, solver status and feasibility diagnostics	21
20. Independent fail-closed validator	22
21. Explanations and controller workflow	23
22. Weekly, monthly and rolling-horizon planning	24
23. What-if simulation and scenario isolation	25
24. KPI definitions and fair evaluation	26
25. Database schema and integrity rules	27
26. Database constraints and representative payloads	29
27. API specification	30
28. Frontend screens and use cases	31
29. Technology stack and deployment	32
30. Security, auditability and nonfunctional requirements	33
31. Functional SRS and traceability	34
32. Automated testing and acceptance criteria	35
33. Numbered implementation roadmap — P0 foundation	36
34. Numbered implementation roadmap — P1 core intelligence	37
35. Numbered implementation roadmap — P2 and P3	38
36. Demonstration scenario and expected evidence	39
37. Risk register and mitigation	40
38. References and evidence register	41
39. Codex Build Contract / Master Implementation Prompt	42

1. Executive summary

RailSync AI coordinates fixed-infrastructure maintenance across Engineering, Traction Distribution (TRD), and Signal & Telecommunication (S&T). Departments record what needs attention, where, how urgently, how long it will take, and which resources and disconnections are required. RailSync combines those requests with a railway network, timetable, train-section occupancy, corridor restrictions, goods forecasts, existing possessions and resource calendars.

Each planning run uses an immutable Integrated Railway Planning State. A transparent baseline planner establishes a fair comparison. A priority engine estimates maintenance importance; a Corridor Availability Engine identifies usable capacity; a Maintenance Opportunity Engine matches demand to windows. Compatibility and resource rules enable eligible work to share a possession. OR-Tools CP-SAT selects among explicit alternatives, after which a separately implemented validator recomputes all hard constraints.

Only a validated, current plan can reach controller approval. Modifications create new revisions and require revalidation. Weekly and monthly views use the same canonical schedule model. Later milestones add rolling-horizon replanning, isolated what-if scenarios and a complete operational interface.

The expected contribution is coordinated, explainable planning with measurable results on declared scenarios. The prototype must demonstrate actual computation, including situations where no feasible plan exists or no improvement is achieved. It must never fabricate optimizer assignments, priority scores, conflict counts or KPI gains.

First build: repository and database, followed by maintenance request creation and validation. Then model infrastructure and traffic, freeze planning snapshots, and produce a tested baseline. A dashboard or trained model must not precede these foundations.

2. Problem interpretation and source status

2.1 Source confidence

The public PS26027 mirrors identify the challenge as “AI-Powered Automatic Block Planning to Maximize Asset Availability for Train Operations on Indian Railways,” under the Ministry of Railways, Software / Transportation & Logistics. Their text agrees on department data integration, prioritization, coordinated blocks and weekly/monthly planning. [S1, S2]

The official SIH portal was attempted on 15 September 2026 but could not be retrieved. Consequently, the wording below is a faithful interpretation of matching public mirrors, not a claim of independently authenticated official text. Obtain and archive the organizer-issued PS before final submission. The proposed engineering details in this document are RailSync design decisions; they are not quoted railway operating rules.

2.2 Exact engineering interpretation

The problem concerns scheduling access to fixed infrastructure for maintenance. It is not rolling-stock servicing. The planning bottleneck is fragmented demand and shared railway capacity: work requests and traffic compete for time and location, while several departments may be able to work within one coordinated possession.

The required integration includes defect and overdue-maintenance records, passenger timetable information, corridor availability and Control Office goods forecasts. Prioritization must reflect criticality, urgency and effects on infrastructure availability. Planning must coordinate departments across weekly and monthly horizons. [S1, S2]

“Automatic” means automated generation of proposals in this prototype. It does not authorize train control, signal operation, traction switching or possession commencement. “Maximize availability” is an objective subject to hard operational constraints and mandatory maintenance; it must not become an incentive to postpone all work.

2.3 Requirement mapping

PS need	RailSync response	Evidence to demonstrate
Maintenance integration	Department forms and TMS/SMMS/TDMS adapters	Import lineage and validated requests
Traffic and corridor integration	Occupancy, COA windows, goods forecasts	Window calculations and source freshness
Priority and coordinated scheduling	Rule/ML priority, compatibility, CP-SAT	Scores, assignments and traceable reasons
Availability and multiple horizons	Comparable KPIs; weekly/monthly plans	Reproducible exports from the same snapshot

3. Assumptions, boundaries and domain decisions

3.1 Prototype scope

Start with one synthetic corridor: four stations, three sections, explicit track identifiers, approximately twelve assets and a handful of trains and requests. Expand only after fixture tests pass. The demonstration target is 10–20 stations, 20–50 sections, 100–200 assets and up to 300 requests. These are test envelopes, not verified capacity limits.

The first scheduler keeps train paths fixed. It finds maintenance inside permitted intervals; it does not cancel, reroute or delay confirmed trains to improve its objective. Proposed future timetable changes require a separate approved model and revalidation of the entire operating plan.

3.2 Explicit assumptions

- All operational timestamps are timezone-aware; persist UTC and display Asia/Kolkata. Timetable imports include service date and day offsets for journeys crossing midnight.
- Use half-open intervals [start, end). Equality at a boundary is allowed only after the configured clearance and setup margins have already been applied.
- Track, direction and electrical isolation coverage are explicit. A section with two tracks is not automatically available when one track is free.
- Work is non-preemptive initially. An interrupted task needs a verified remaining-duration update; the system never assumes it can resume without setup.
- Missing critical traffic coverage, isolation mapping, rule configuration or resource eligibility blocks approval. Missing optional ML features can invoke a disclosed rule fallback.
- Configured buffers and compatibility rules require domain review. Values in fixtures are illustrative, not official Indian Railways safety standards.

3.3 Outside the approved implementation scope

Live connections to railway systems, certified interlocking simulation, automatic possession grants, live traction control, nationwide deployment, and predictive claims trained only on fabricated labels are outside this prototype. The name Integrated Railway Planning State describes a versioned planning representation; no full digital-twin claim is made.

3.4 Decisions needing domain confirmation

Record owner, due date and status for: official PS copy; department field dictionaries; authoritative COA window semantics; track/headway rules; electrical isolation topology; work compatibility; mandatory defect deadlines; crew travel/rest policy; freshness thresholds; and approval roles. Unknown safety-critical settings cause fail-closed behavior, not guessed defaults.

4. Research-informed design rationale

The following are bounded design inferences from research, not replications of the papers or promises of their results.

Source	Research finding or scope	RailSync design inference	Limitation
R1: Lidén & Joborn, 2017	Integrated planning of traffic and maintenance windows with spatial/temporal capacity representation	Put traffic and maintenance into one snapshot and model possession footprints explicitly	RailSync initially keeps traffic fixed; it does not implement the paper's joint tactical model
R2: Resource considerations, 2018	Maintenance planning includes crew availability and working-time restrictions	Treat calendars and resource feasibility as hard constraints	Local railway crew policy must be supplied independently
R3: Maintenance possession scheduler, 2023	A case-specific possession scheduling model is formulated and experimentally evaluated	Begin with a bounded corridor and test actual schedule outputs	Published runtimes and quality do not transfer to RailSync
R4: Integrated timetabling/platforming/maintenance, 2022	Joint infrastructure-use decisions and time-window search are studied	Keep a future path to coupled traffic scenarios and candidate refinement	Full platform and train-path optimization is beyond the first model
R5: XGBoost, 2016	A scalable tree-boosting system for supervised learning	Evaluate tabular maintenance models against a transparent baseline	No evidence yet establishes railway prediction quality
R6: SHAP, 2017	Additive feature attribution explains individual model predictions	Store feature contributions alongside the model output	Attribution is not causality or a safety certificate

Research supports coordination and explicit constraints. RailSync's architecture, milestone ordering, API contracts and validation gates are original implementation choices. Numerical research improvements are intentionally not reused as RailSync results.

The first implementation favors a fixed-timetable candidate-selection model because its decisions are easy to inspect, independently validate and test by exhaustive enumeration on small instances. Greater model complexity is justified only by a demonstrated failure of the simpler model to represent required work.

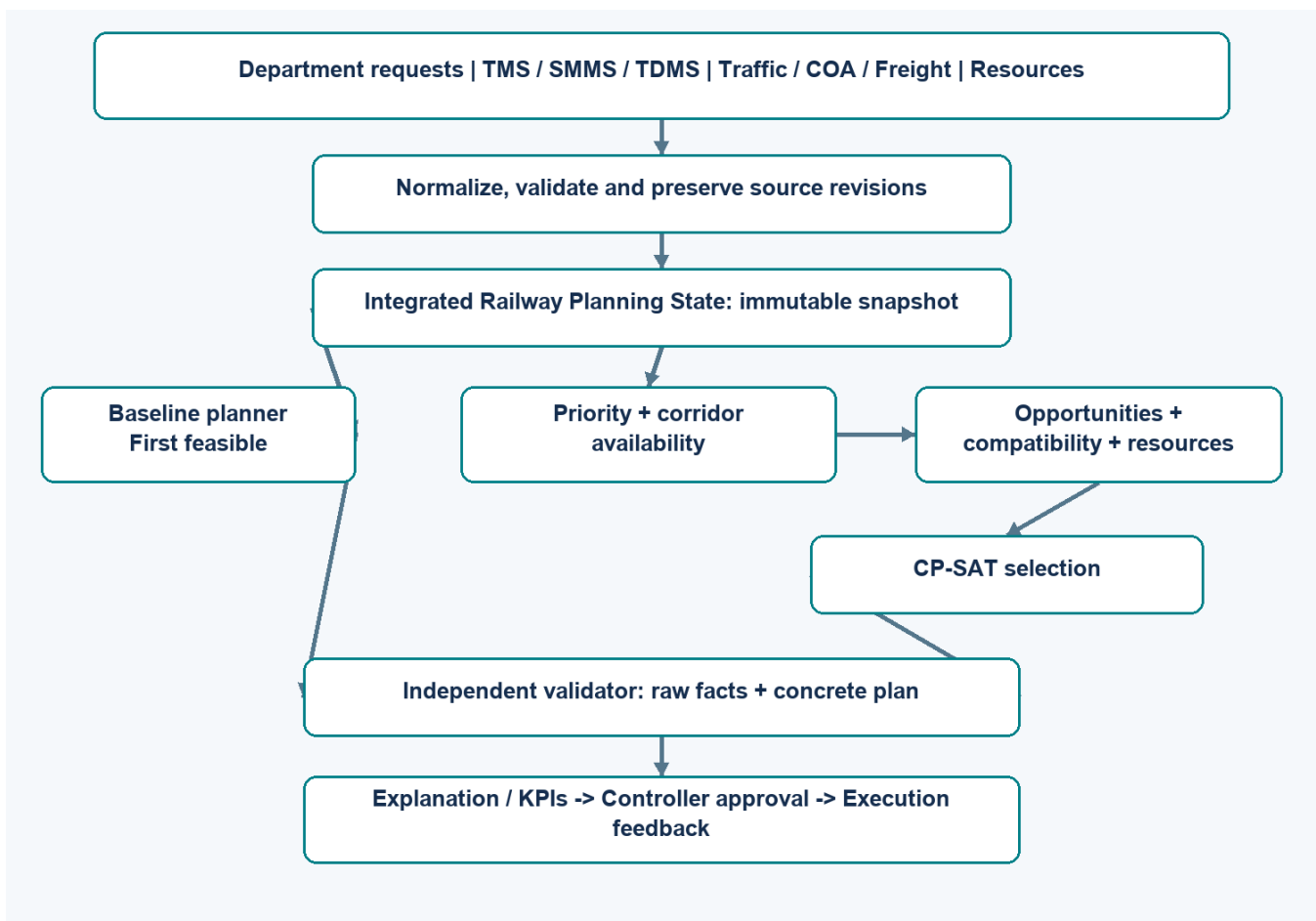
5. AI, ML, optimization and validation

Component	Method	Answers	Does not establish
AI decision support	Overall combination of inference, search and explanations	What proposal should a controller consider?	Authority to execute railway operations
Rule priority baseline	Versioned deterministic formula	Which requests rank higher under declared policy?	A learned failure probability
ML priority/risk	Random Forest or XGBoost, only after label review	Estimated urgency ranking or probability of a defined event	Exact safe start/end times
SHAP	Model feature attribution	Which features contributed to this prediction?	A causal diagnosis or CP-SAT decision explanation
CP-SAT	Constraint programming / integer optimization	Which feasible candidates and resource assignments best meet the objective?	Feasibility outside the encoded model
Deterministic validator	Independent interval and policy checks	Does this concrete plan satisfy the declared constraints?	Railway certification or unknown physical conditions
Controller	Human decision and operational procedure	Approve, modify, reject, or request replanning?	Permission to bypass a failed validator

Machine learning is optional at runtime when defensible training data is unavailable. A working rule baseline plus genuine optimization is preferable to an unvalidated model presented as operational intelligence. Every score carries score_method, rule/model version, feature timestamp and explanation.

An artificial neural network (ANN) is an optional future forecasting experiment for freight demand, delay, congestion or maintenance demand. It must beat a simple forecast baseline on held-out data before use. It never becomes the final scheduler. A language model is unnecessary for core functionality; explanations are generated from structured facts using templates.

6. Final logical architecture



Architecture — RailSync proposed design

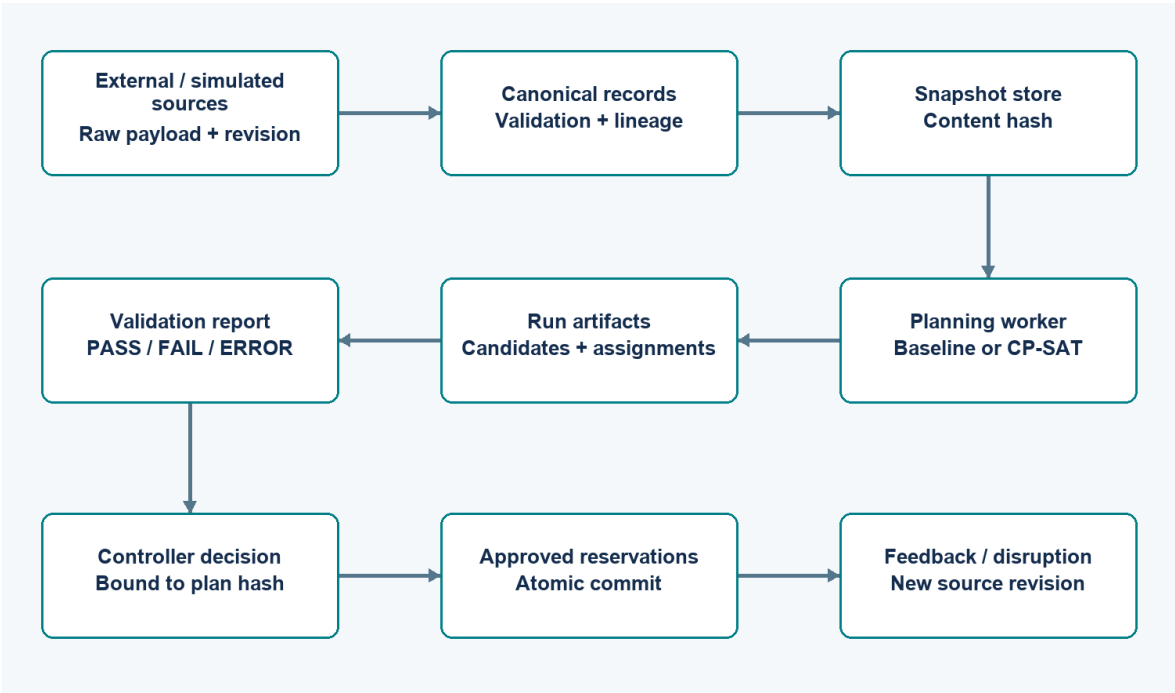
The application is a modular Python backend with a separate worker process for planning jobs, a PostgreSQL database and a web frontend. Modules communicate through typed domain objects, not through copied UI state. A worker reads a snapshot once and writes versioned run artifacts.

There are two planning branches. The baseline generates deterministic first-feasible assignments without bundling. The optimized branch performs prioritization, availability, opportunity generation, compatibility/resource assessment and CP-SAT selection. Both produce the same PlanRevision contract and are evaluated by the same independent validator and KPI definitions.

The validator reads raw snapshot facts and concrete assignments. It does not accept “feasible=true” from candidate generation. Invalid plans remain inspectable as rejected computation artifacts but cannot be approved or exported as approved schedules.

The controller workflow sits after validation. A runtime data change marks affected plans stale, creates a new snapshot and later triggers replanning. Approval binds to the exact plan content hash, validation report and operational-state revision. Execution feedback creates new facts; it does not rewrite the historical snapshot.

7. Data flow and module contracts



Dataflow — RailSync proposed design

Module	Input	Output	Failure behavior
Ingestion	Forms, CSV/JSON, source mappings	Normalized revisions, error report	Quarantine malformed rows
State builder	Versioned source facts and horizon	Snapshot manifest and hash	Reject incomplete critical coverage
Priority	Eligible requests, feature snapshot	Scores with reasons	Disclosed rule fallback or reject missing mandatory fields
Availability	Occupancy, COA, restrictions, buffers	Per-footprint windows	Unknown capacity remains unavailable
Opportunity	Requests, windows, duration and policy	Explicit candidate set and exclusions	Empty set with reason codes
Compatibility/resources	Candidates, work rules, calendars	Valid bundle alternatives, resource allocations	Exclude unknown/incompatible combinations
Optimizer	Candidate set and objective configuration	Selected candidates, status, bound	No assignments for absent solution
Validator	Raw snapshot and selected intervals	PASS, FAIL or ERROR report	Only PASS permits review
Decision/KPI	Plan, facts, run metadata	Reasons, comparison and approval record	Preserve missing/undefined metrics

All outputs carry `snapshot_id`, `schema_version` and `producer_version`. Processing records duration, status, input counts and output counts. No module uses a new live database read halfway through a run to silently change its inputs. Cancellation and failure leave an auditable job status.

8. Maintenance request system and ingestion

8.1 User-facing request contract

A `MaintenanceRequest` describes demand before a time slot is chosen. Required fields are `department`, `asset_id` or explicit footprint, `issue_type`, `severity`, `urgency`, `asset impact`, `estimated work minutes`, `block requirement`, `power/disconnection requirement`, `earliest start`, `deadline policy`, `required skills/equipment` and `supporting description`. The department supplies duration and constraints; the planner selects final times.

Departments are ENGINEERING, TRD and SNT. A request may span several tracks or sections through `request_footprint` rows. Validate positive duration, valid ranges, known assets, location consistency, power requirements and allowed department ownership. Return field-level errors; never silently coerce an unknown section into a default corridor.

8.2 Source adapters

TMS means Track Management System; SMMS means Signalling Maintenance & Management System; TDMS means Traction Distribution Management System in the retrieved PS. COA is Control Office Application. BDMS is retained as the named request/block workflow system without inventing its integration API. [S1, S2]

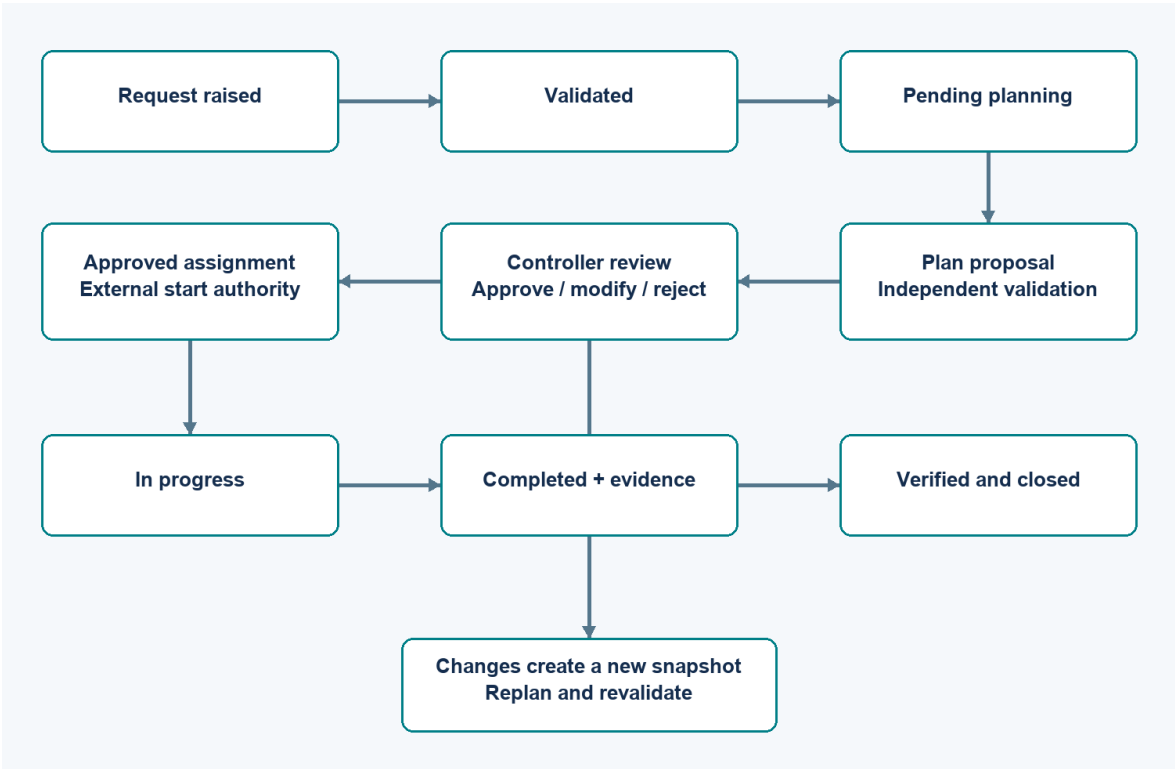
Implement explicitly labeled SIMULATED and IMPORTED adapters. Each source record has `source_system`, `external_id`, `source_revision`, `event_time`, `received_at`, `schema_version`, `payload_hash` and `import_batch_id`. Unique source/revision keys make retries idempotent. A newer source revision creates a new canonical revision; conflicting or out-of-order updates are held for review.

CSV/JSON mapping is a project-defined contract, not an assertion about real railway interfaces. Preview import results before committing valid rows; quarantine invalid rows with row number, error code and original value. Preserve raw payloads with restricted access. Source deletions use tombstones; they cannot remove historical evidence.

8.3 Minimum backend evidence

Automated tests cover create/read/update, department authorization, invalid duration, unknown asset, duplicate import, partial batch failure and concurrent update rejection. A request is eligible for planning only after required validations and a transition to `PENDING_PLANNING`.

9. Lifecycle, actors and state transitions



Lifecycle — RailSync proposed design

Maintain request state separately from plan state. Request states are RAISED, VALIDATED, PENDING_PLANNING, IN_PROGRESS, COMPLETED and CLOSED, with DEFERRED, REJECTED and CANCELLED branches. Priority is a computed assessment; proposed/approved/scheduled membership belongs to plan assignments and is shown as a derived request status. This prevents two scenario plans from corrupting one shared request lifecycle.

PlanRevision states are DRAFT, SOLVING, VALIDATING, READY_FOR_REVIEW, APPROVED, REJECTED, SUPERSEDED, STALE and FAILED. A plan may contain deferred requests while its selected assignments pass validation. An infeasible mandatory-work problem is FAILED with reasons, not a successful empty plan.

Actor	Permitted actions	Controls
Department user	Create and revise owned requests; submit completion evidence	No approval of plan revisions
Planner	Build snapshots, run planners, inspect alternatives, propose modifications	Cannot alter immutable run outputs
Controller	Approve, reject, request replan; record authorized execution state	Requires current PASS report and reason
Administrator	Manage users, adapters and versioned configuration	No implicit controller privilege
Auditor	Read plans, provenance and decision history	Read-only access

Transitions are commands with actor, expected_version, reason and timestamp. Completion requires actual start/end and completion evidence. Closure is department verification, not automatic upon reaching a planned end time. Rejection of a plan returns unassigned requests to planning eligibility; it does not reject the maintenance need itself.

10. Network, assets and operating restrictions

Represent Station, RailwaySection, Track and Asset separately. A section joins two stations; tracks carry permitted direction, operating status and identifiers. Assets reference a footprint that may cover multiple tracks. Electrification zones map to every affected track, including adjacent sections when required by the supplied isolation topology.

Store speed limits and running-time assumptions as source facts. Do not compute train occupancy from distance and maximum speed alone and call it authoritative. Synthetic running times must include declared acceleration/dwell assumptions and carry SIMULATED provenance.

Restrictions have a typed scope, effective interval, track/zone coverage and rule version. Examples are existing possession, infrastructure closure, speed restriction, prohibited work time, electrical isolation unavailability, access restriction and weather-related work prohibition. A speed restriction modifies occupancy through a reviewed traffic transformation; it is not automatically a complete closure.

Prototype default: a possession reserves exclusive use of its full track footprint. Partial capacity sharing requires explicit permitted-capacity semantics and new tests before enablement. Work near an adjacent live line remains prohibited unless a reviewed rule allows it. Signalling disconnection and electrical isolation are separate from track possession; each has its own dependencies and restoration duration.

Acceptance fixtures include single track in both directions, independent double tracks, a possession affecting both tracks, multi-section electrical coverage, and an asset moved by a newer revision. Historical plans continue to resolve the old asset/footprint revision through their snapshot.

11. Timetable, occupancy and goods forecasts

11.1 Train representation

Train identifies a service; TrainRun identifies one dated movement. TrainSectionOccupancy stores run_id, track_id, route_sequence, enter_at, exit_at, timing_source, uncertainty and version. Every movement in the horizon must have a contiguous, valid route or an explicit coverage gap.

Expand repeating timetable services into dated runs. Preserve midnight day offsets. Sort occupancy by route sequence and reject exit before enter, disconnected paths and unexplained temporal overlap between incompatible route segments. Station/platform occupations can be added as capacity resources where the modeled corridor requires them.

For interval arithmetic, convert to integer minutes relative to the snapshot horizon. Round occupied intervals outward and available intervals inward. Round duration requirements up. Record discretization so the validator reproduces conservative timing without trusting optimizer calculations.

11.2 Freight representation

Confirmed freight movements are hard occupancy just like other confirmed trains. Uncertain goods forecasts contain forecast_id, issue_time, validity, corridor/track scope, interval, expected_count, confidence/probability semantics, source and scenario_id. A confidence of 0.7 is not automatically a calibrated 70% train-arrival probability.

Default demonstration policy reserves the complete declared uncertainty envelope for unconfirmed freight when constructing approvable windows. A less conservative expected-impact scenario may be shown as experimental, but it cannot be approved unless its uncertainty treatment satisfies a controller-configured policy. No penalty term may authorize overlap with confirmed occupancy.

Risk exposure can be estimated as expected_count multiplied by the overlap fraction of a forecast interval, if a uniform-within-window assumption is explicitly declared. Preserve scenario-level route correlation; do not sum the same train across sections and label it unique trains affected.

11.3 Freshness and missing data

Source-specific maximum age and horizon coverage are versioned settings. Missing coverage is UNKNOWN, not zero traffic. Expired operational data marks dependent plans stale. The monthly view labels far-future uncertainty and cannot imply the precision of near-term confirmed movements.

12. COA and Corridor Availability Engine

COA availability is an input assertion with provenance, not a replacement for occupancy checks. If an authoritative allowed window exists, intersect it with internally derived capacity. If it conflicts with occupancy or restrictions, emit `SOURCE_CONFLICT` and prohibit use pending resolution. The prototype can derive simulated COA windows, but must label that derivation.

For each required track footprint:

- Expand confirmed occupancy with configured entry/exit clearance margins.
- Add protected freight envelopes, existing possessions and hard restrictions.
- Union overlapping forbidden intervals, clipped to the horizon.
- Subtract them from the declared operating/COA allowed intervals.
- Intersect the resulting free intervals across all tracks and zones required by the work.
- Filter by usable duration after work-specific setup and restoration; preserve the reasons for all exclusions.

CandidateWindow includes footprint, start/end, provenance IDs, raw duration, margin policy, uncertainty policy and limiting boundaries. Generic availability must not deduct task setup twice; choose one convention: windows already exclude train-clearance margins, while opportunities add work setup/restoration within the window.

Synthetic boundary example: a train exits at 01:50 and the next enters at 03:20. With five-minute clearance on each side, the raw usable interval is 01:55–03:15. A work package requiring 10 minutes setup, 60 minutes work and 10 minutes restoration fits exactly in 80 minutes. A package requiring 81 minutes does not.

Tests cover overlapping occupancies, empty/full horizons, boundary equality, multi-track intersections, missing COA coverage, midnight, and adjacent electrical-zone restrictions. The engine must be independent of priority: a high score cannot manufacture capacity.

13. Integrated Railway Planning State

A PlanningSnapshot is an immutable manifest containing the planning horizon; network and asset revisions; eligible request revisions; timetable/occupancy; forecast set; resource calendars; restrictions; approved commitments; disruptions; rules; feature cutoff; scenario lineage; and generator seed where applicable.

Build snapshots in a consistent database transaction. Materialize their canonical payload or reference immutable revision rows. A snapshot that simply stores mutable foreign keys is insufficient. Canonicalize ordering and timestamps before computing a SHA-256 content hash. The snapshot also records per-source watermarks and completeness results.

PlanningRun references snapshot_id, planner_type, code_commit, dependency-lock hash, objective_config hash, model version, candidate-generator version, validator version, solver version, seed, worker count and time limit. Its artifacts include priority assessments, windows, candidates, exclusions, solution, validation report and KPI inputs.

A new import never mutates a run's snapshot. The system compares operational revisions to determine staleness, then creates a replacement snapshot. What-if scenarios reference a parent snapshot and explicit patch set, preserving provenance of each changed fact.

Required tests prove that source changes after snapshot creation cannot change its hash or planner inputs; identical canonical facts produce identical hashes; critical missing data rejects creation of an approvable snapshot; and the same stored artifact can reconstruct a plan after a service restart.

Exact solver tie choices can vary with versions and parallel search. Reproducibility means preserving inputs and settings; deterministic CI uses a pinned solver and one worker, and compares feasibility/objective where several equal optima exist.

14. Baseline planner specification

The baseline is a declared first-feasible heuristic representing uncoordinated planning behavior for comparison. It is not a measured model of current railway practice. It shares the same traffic, resource and operational rules as RailSync.

Sort requests by mandatory status, then deadline, created_at and stable ID. For each request, enumerate all single-request candidates in chronological order and select the first one that fits the still-available resources and footprint. Do not bundle or rearrange earlier choices. If none fits, record a deferral reason. Reserve selected resources immediately. Mandatory work that cannot be allocated makes the result non-approvable.

At P0, implement a minimal tested interval and resource feasibility library for this first-feasible planner. P1 extends availability and compatibility; it must not make the baseline artificially weaker by withholding feasible single-request alternatives. Re-run the baseline on the same expanded candidate universe when comparing the P1 optimizer.

Both planners produce PlanRevision with selected assignments, unassigned reasons and run metadata. The independent P1 validator must later validate baseline outputs too. Before that validator exists, baseline plans remain development artifacts and cannot reach approval.

Automated evidence includes stable ordering, exact expected assignment on a tiny fixture, no train overlap, no crew double booking, explicit no-window reasons and a case where first-feasible is worse than global selection. The latter is a constructed regression fixture, not a claimed real-world improvement.

Do not deliberately inject avoidable conflicts, extra downtime or random inefficiency into the baseline. Report equal or worse RailSync results honestly and include whether each planner met the same mandatory-work requirements.

15. Priority and risk engine

15.1 Transparent rule baseline

Define normalized features S =severity, U =urgency, C =asset criticality, O =overdue fraction capped at one, and I =declared operational impact, each in $[0,1]$. An illustrative initial policy is $\text{score} = 100 \cdot (0.30S + 0.25U + 0.20C + 0.15O + 0.10I)$. Store each contribution and policy version. This weighting is a configurable project assumption, not official policy. Missing mandatory features require correction; do not convert missing values into low priority.

Safety-critical or mandatory-by-deadline classification is a reviewed rule independent of the score. Neither a low prediction nor a controller preference can downgrade a hard maintenance obligation through objective weights.

15.2 ML experiment and labels

Choose a precisely defined target before training: for example, probability of a verified adverse asset event within seven days if unresolved, or expert-labeled urgency class. These are different tasks and require different labels. Do not train on rule-generated labels and describe the result as independently learned failure risk.

Candidate features include pre-cutoff condition, defect type, recurrence, failure history, severity, overdue duration, asset criticality and traffic exposure. Exclude future completion data, later inspection findings, outcome-derived fields and identifiers that leak labels. Maintenance intervention changes observed outcomes; document selection bias and censoring rather than treating every serviced asset with no failure as an untreated negative.

Evaluate Random Forest and XGBoost against the rule baseline using a temporal holdout and grouped asset/section separation where feasible. Report dataset size, label provenance, class balance, PR-AUC and recall for rare-event classification, calibration/Brier score for probabilities, and ranking metrics when the target is a ranking. Report uncertainty and subgroup results. [R5, T5]

15.3 Promotion and explanation

Promotion requires documented label validity, no leakage, better held-out performance on the chosen primary metric, acceptable critical-case recall and domain review. Exact thresholds must be recorded before training. Without this evidence, keep RULE as production default and mark the model EXPERIMENTAL.

SHAP explanations record background dataset version, feature vector, base value, contributions and output scale. Contributions in log-odds must not be presented as percentage-point increases. Plain-language templates summarize the strongest factors without claiming causality. [R6]

16. Maintenance Opportunity Engine

An opportunity is a concrete way to do work inside a capacity window. It is not a score alone. Inputs are request revisions, priority assessments, windows, setup/restoration policy, footprint and resource requirements. Outputs include candidate intervals, work subintervals, eligible resources, benefit/cost terms and rejection evidence.

For a single task with work duration d , setup a and restoration b , require candidate length at least $a+d+b$. Enforce earliest start on the work start, deadlines on work completion or restoration as explicitly configured, and horizon containment on the entire possession. Preserve both work and possession times to avoid ambiguous deadline checking.

Initially enumerate integer-minute start alternatives on a configured grid, including the earliest and latest feasible starts. The grid, candidate cap and pruning strategy are run metadata. Pruning must never be hidden: a solver result is optimal only over the generated candidate set. If no candidate is found, distinguish `WINDOW_TOO_SHORT` from `SEARCH_LIMIT_REACHED`.

Candidate costs are computed facts: reserved track-minutes, expected forecast exposure under declared assumptions, crew movement effort and deviation from an existing commitment. Candidate benefit sums selected request utility once per request. An opportunity score used for ordering does not decide feasibility.

Keep feasible single-task candidates even when bundle generation is enabled. Prune only proven-dominated alternatives with equivalent request coverage, resource use, temporal effects and footprint, or label heuristic pruning explicitly. Store counts before and after pruning.

Required fixtures include a job longer than every gap, exact fit, earliest-start clipping, restoration past deadline, no eligible crew, a multi-section request and a candidate excluded solely by search limits. These reasons feed explanations and diagnostic screens.

17. Cross-department bundling and resources

17.1 Compatibility policy

Pairwise compatibility is a versioned matrix keyed by work type, footprint relation, power state and disconnection requirements. Values are ALLOW_PARALLEL, ALLOW_SEQUENTIAL, FORBID or UNKNOWN. UNKNOWN cannot be bundled. Department identity alone does not establish compatibility.

An Engineering task and TRD task can share a possession only when track coverage, isolation sequence, access, equipment clearance and restoration requirements agree. An S&T test requiring energized equipment cannot be assumed concurrent with a power-off activity. Group-level checks must also catch hazards not captured by pairwise permission.

For compatible parallel work, block length is shared setup + maximum task completion offset + shared restoration. For sequential work, use the precedence schedule's makespan, not the maximum individual duration. Record each task's actual interval and each setup/restore stage. Shared setup is applied only where the policy explicitly permits it.

17.2 Resource engine

Resource pools describe skill/equipment types and capacity calendars. Named ResourceUnits support concrete crew/machine assignments. Eligibility considers qualifications, expiry, department permissions, location, calendar, travel, work/rest limits and incompatible duties. Consume resources during setup and restoration where required.

For the initial fixed-candidate model, enumerate eligible resource assignments into candidate alternatives and reject those violating calendars or intra-bundle capacity. Pairwise incompatibility handles insufficient travel time between selected alternatives; cumulative capacity checks handle pooled units. Prevent combinatorial explosion with declared caps and diagnostics.

Tests include two individually feasible tasks that exceed shared capacity, qualified versus unqualified crews, travel between sections, rest across midnight, mutually incompatible electrical states and triple combinations that violate a group-level rule. Missing travel or qualification data means no eligible assignment until supplied.

18. CP-SAT mathematical model

18.1 Implementable first model

Use finite, fully timed candidate alternatives. Each candidate represents one possession containing one or more tasks, explicit task subintervals, footprint, setup/restoration and a concrete eligible resource allocation. CP-SAT selects candidates; it does not invent times outside this candidate set. Optional interval variables can express their occupancy. This is a deliberate first implementation of scheduling. [T1, T2]

Sets: I=requests; C=candidate alternatives; C_i=candidates containing i; M=mandatory requests; E=mutually incompatible candidate pairs; R=resource pools; K=event intervals induced by all candidate starts/ends and calendar changes.

Parameters: A_{ic} is 1 if candidate c contains i; L_c is reserved track-minutes; F_c is forecast exposure cost; V_c is travel cost; Q_c is commitment-change cost; U_i is integer maintenance utility; a_{crk} is candidate resource use during event interval k; cap_{rk} is available capacity. All objective coefficients are bounded integers with recorded scale factors.

Decision variables: x_c in {0,1} selects candidate c; y_i in {0,1} indicates scheduled request i; u_i=1-y_i indicates deferred request. Optional interval presence equals x_c. An accepted previous assignment has a retained-candidate indicator; changes are measured against that specific assignment, not simply task inclusion.

18.2 Hard constraints

1. Coverage: $\sum(c \in C_i) x_c = y_i$ for every request. This forbids duplicate work across overlapping bundles.
2. Mandatory work: $y_i=1$ for every mandatory request whose obligation falls within this horizon. Candidate construction already enforces its hard deadline. An empty C_i makes the model infeasible.
3. Incompatibility: $x_c + x_d \leq 1$ for every (c,d) in E. E includes conflicting track possessions, contradictory isolation/disconnection states, named-resource overlap and insufficient travel/rest separation.
4. Pooled resources: $\sum(c) a_{crk} x_c \leq cap_{rk}$ for each resource and event interval. Calendars and changes in capacity are explicit.
5. Existing fixed possessions and in-progress work are either immutable reservations during candidate generation or forced retained candidates, never both in a way that double-counts them.
6. Precedence: prohibit candidate pairs that violate request dependencies; if i requires j, enforce $y_i \leq y_j$ unless j is already verified completed. Required finish/start lags are included.
7. All selected candidates must satisfy the snapshot horizon, footprint, occupancy, isolation, policy and duration predicates. Construct only eligible candidates, and independently recheck after solving.

For an interval-based later extension, use optional task intervals, NoOverlap for exclusive resources and Cumulative for pools; retain all same validation obligations. Do not mix a partially implemented variable-time model with fixed candidate cost assumptions.

19. Objectives, solver status and feasibility diagnostics

19.1 Configurable objective

Default objective minimizes:

$$J = wU * \sum(i) U_i * u_i + wL * \sum(c) L_c * x_c + wF * \sum(c) F_c * x_c + wV * \sum(c) V_c * x_c + wQ * \sum(c) Q_c * x_c.$$

The terms mean deferred maintenance utility, possession exposure, uncertain freight exposure, resource movement and disruption of previous commitments. Normalize costs to documented scales, then convert to safe bounded integer coefficients. Log each term separately. Do not call priority-weighted completion a physical reliability improvement.

Mandatory deadlines, confirmed train separation and resource feasibility are hard constraints and never weighted penalties. To avoid selecting no optional work merely to minimize downtime, set and test utility weights explicitly, or use two passes: maximize scheduled utility, then minimize operating costs with that utility fixed. If pass one is only FEASIBLE, say pass two preserves its incumbent utility, not a proven global optimum.

Weights are scenario configuration under planner/controller policy, versioned and shown in the UI. Run sensitivity tests for at least low/medium/high downtime and freight penalties. Avoid double counting both bundle bonuses and setup savings when the possession cost already captures the same benefit.

19.2 Status contract

OPTIMAL means optimal for the encoded candidate model. FEASIBLE means a solution exists without proof of optimality. INFEASIBLE means no solution satisfies that model. UNKNOWN means the solver did not establish a solution/status within limits. MODEL_INVALID is a construction failure. Only actual FEASIBLE/OPTIMAL assignments are sent to validation. [T1]

Store wall time, objective, best bound, relative gap where defined, candidate counts, seed, worker count and model validation output. A timeout with no incumbent returns no schedule. An empty optional-work solution must be labeled as such; it is not a maintenance success.

19.3 Diagnostics

Preserve candidate exclusion reasons. For mandatory infeasibility, show requests with no candidates and conflicting hard commitments. Optional assumption literals can help diagnose conflicting requirement groups, but explanations must distinguish a sufficient conflict set from a proven minimal cause. A deferred request with feasible candidates may simply lose under the objective; label it NOT_SELECTED_UNDER_OBJECTIVE unless a tested counterfactual establishes a stronger reason.

20. Independent fail-closed validator

The validator is a separate backend module and test suite. It reads concrete assignment times, task coverage and resource allocation against raw snapshot facts. It must not call the optimizer's candidate-feasibility function or trust cached overlap flags. Sharing immutable schema types is acceptable; sharing the same unchecked feasibility implementation defeats independence.

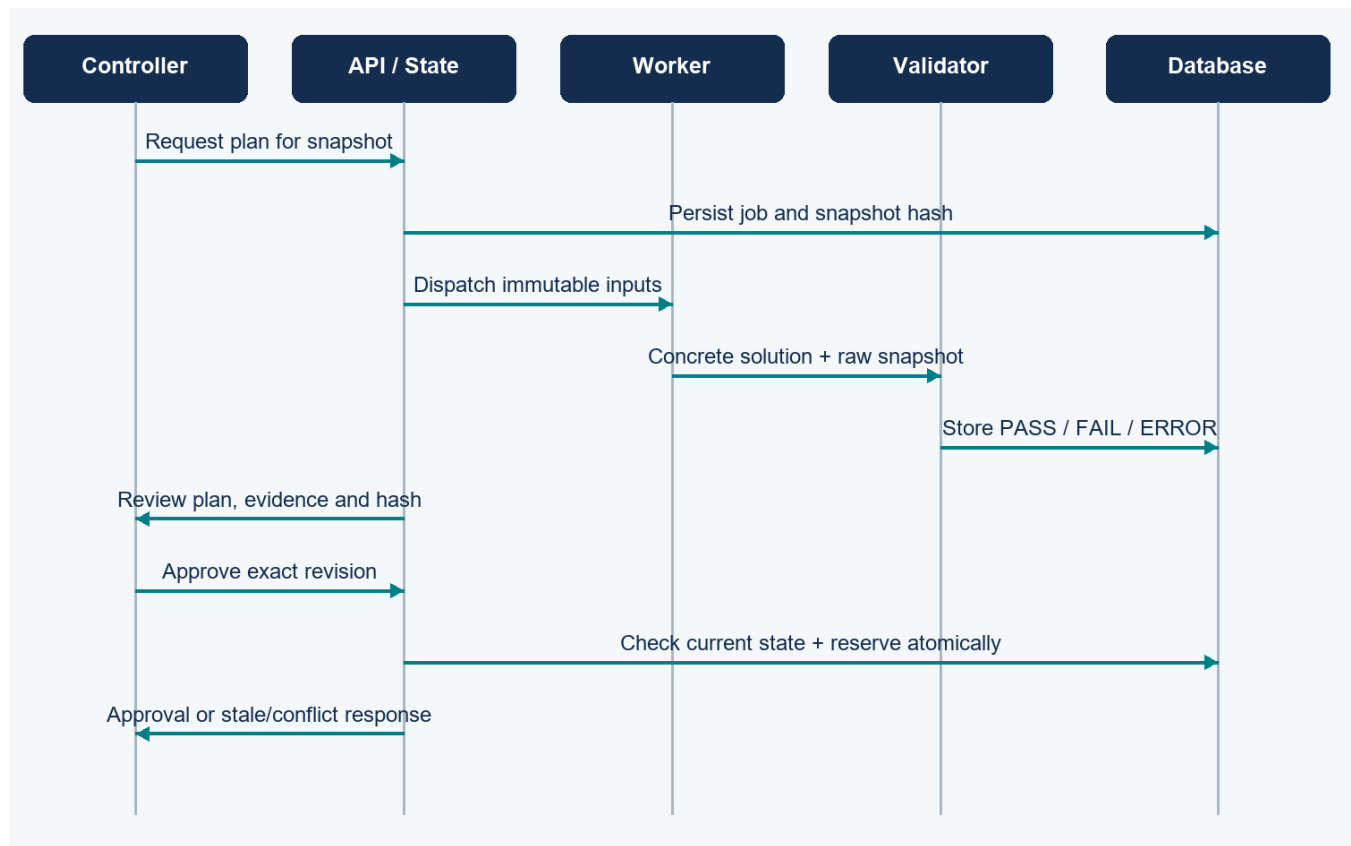
Check	Required evidence	Failure code example
Identity and provenance	Snapshot hash, request revisions, rule version	SNAPSHOT_MISMATCH
Task coverage	No duplicate task; all mandatory obligations met	DUPLICATE_TASK / MANDATORY_UNSCHEDULED
Time and duration	Exact setup/work/restore, earliest/deadline, horizon	DURATION_SHORT / DEADLINE_VIOLATION
Infrastructure	Track footprint, closures, adjacent coverage	TRACK_CONFLICT
Train clearance	Buffered raw occupancy and selected possessions	TRAIN_OVERLAP
Electrical/signalling	Correct isolation coverage and restoration sequence	ISOLATION_MISSING
Work compatibility	Pairwise and group-level rules	WORK_INCOMPATIBLE
Resources	Qualification, capacity, calendar, travel, rest	RESOURCE_CONFLICT
Commitments	Frozen and completed work preserved	FROZEN_WORK_CHANGED
Data quality	Coverage, freshness and no unresolved critical conflict	DATA_STALE / COVERAGE_UNKNOWN

A report includes report_id, plan_hash, snapshot_hash, validator_version, checked_at, per-check result, evidence IDs and status PASS/FAIL/ERROR. Exceptions, timeouts, missing data or unsupported rule types return ERROR and block approval. They must never default to PASS.

Revalidate on every plan modification and before approval against the current operational revision. Approval freshness is transactional: if the source revision changes between checking and commit, return a conflict and regenerate/revalidate. Keep a pre-execution freshness check in the simulated workflow; real possession authority remains external.

Mutation tests deliberately shorten duration, move a block into a train path, remove a crew, duplicate work, omit an isolation, alter a snapshot hash and corrupt a rule version. Each must fail with a traceable reason. Validator failures are engineering defects or invalid proposals, not conflicts to hide from the demo.

21. Explanations and controller workflow



Sequence — RailSync proposed design

DecisionExplanation is a structured record: request_id, outcome, priority method/reasons, selected window facts, compatible co-work, resource assignment, binding restrictions, objective contributions and alternate-candidate evidence. Templates convert it into readable text. A model attribution explains priority; a schedule explanation cites the actual candidate and constraints.

Example template: “Selected in window W because the 80-minute possession fits the available footprint, crew R is eligible, and compatible work shares the setup. Rule priority is 74 under policy v1.” This is a template, not an actual RailSync result. A deferred item shows the actual reason code and any verified future candidate; never invent a next feasible date.

Approval command requires controller role, plan revision, content hash, PASS report, expected operational revision and reason. In one transaction, recheck staleness and active commitments, reserve approved assignments and write an append-only decision event. Two controllers racing to approve conflicting plans cannot both succeed.

Modify creates a child revision from structured edits such as pinned start, excluded request or resource replacement. It triggers fresh solving where necessary and always fresh validation. A failed modification remains rejected; the old approved plan is preserved unless explicitly superseded through the controlled workflow.

Reject records a reason and leaves the maintenance need eligible for future planning. Replan creates a new run using current state. APPROVED indicates approval of the software proposal, not a real-world possession grant. The execution screen records externally authorized start, handback and completion in the prototype.

22. Weekly, monthly and rolling-horizon planning

Weekly planning uses an explicit start/end interval, typically seven local-calendar days. Monthly planning uses a defined calendar month, not an assumed 30 days. Both expand to minute-based intervals and share the same plan entities, constraints and validator.

The monthly view distinguishes firm near-term commitments from tentative future proposals. Forecast validity limits are visible. If the full monthly candidate model is too large, solve overlapping subhorizons with frozen commitments and shared resource carryover. Label this a heuristic decomposition; do not claim global monthly optimality. Check cross-boundary travel, rest, setup and tasks spanning midnight or week boundaries.

Rolling-horizon replanning starts from a new event: updated occupancy, urgent request, resource outage, restriction or changed forecast. Record event time and ingestion time. Deduplicate events, debounce rapid changes and supersede obsolete queued runs. A worker finishing an older run cannot overwrite the active plan.

Freeze completed and in-progress work. Preserve approved assignments within a configurable freeze interval unless an operational conflict makes them invalid; such conflicts require controller attention, not silent movement. Replan future eligible work with a change penalty and explicit remaining-duration facts for interrupted work.

Output is a plan difference: unchanged, moved, newly scheduled, removed and newly deferred assignments, with explanations. The replacement must pass validation and approval. A newly unsafe old plan becomes STALE/REQUIRES_ATTENTION even if no replacement is feasible; retaining a visible old plan must not imply it is still executable.

Tests include month-end, leap day, resource rest across horizons, disruption during work, repeated identical event, worker race and an infeasible replacement with mandatory work.

23. What-if simulation and scenario isolation

A scenario is an immutable parent snapshot plus a validated patch set. Supported patches initially include train delay, forecast increase, resource outage and new urgent request. Apply a train delay through its downstream route occupancy consistently; changing one section alone can create an impossible train trajectory.

Scenario records contain `parent_id`, patch schema, actor, seed, description and creation timestamp. The scenario builder validates resulting traffic consistency before planning. What-if runs use the same engines and rules as normal runs, with clearly marked SIMULATION status and no approval side effects on the operational plan.

Compare baseline and optimized runs within each scenario, then compare each scenario with the parent using the same KPI definitions. Display assumptions and changed records. Report infeasible outcomes and runtime; do not always steer the scenario to a visually improved schedule.

Promotion is not a database copy of scenario assignments into production. An accepted idea must become an authorized source update, followed by a fresh snapshot, re-solve, validation and controller approval. This protects against stale hypothetical conditions and conflicting commitments.

Minimum tests prove parent immutability, deterministic patch application, invalid delay rejection, no live reservations from simulation, identical rules between scenario and operational runs, and correct lineage in exports.

24. KPI definitions and fair evaluation

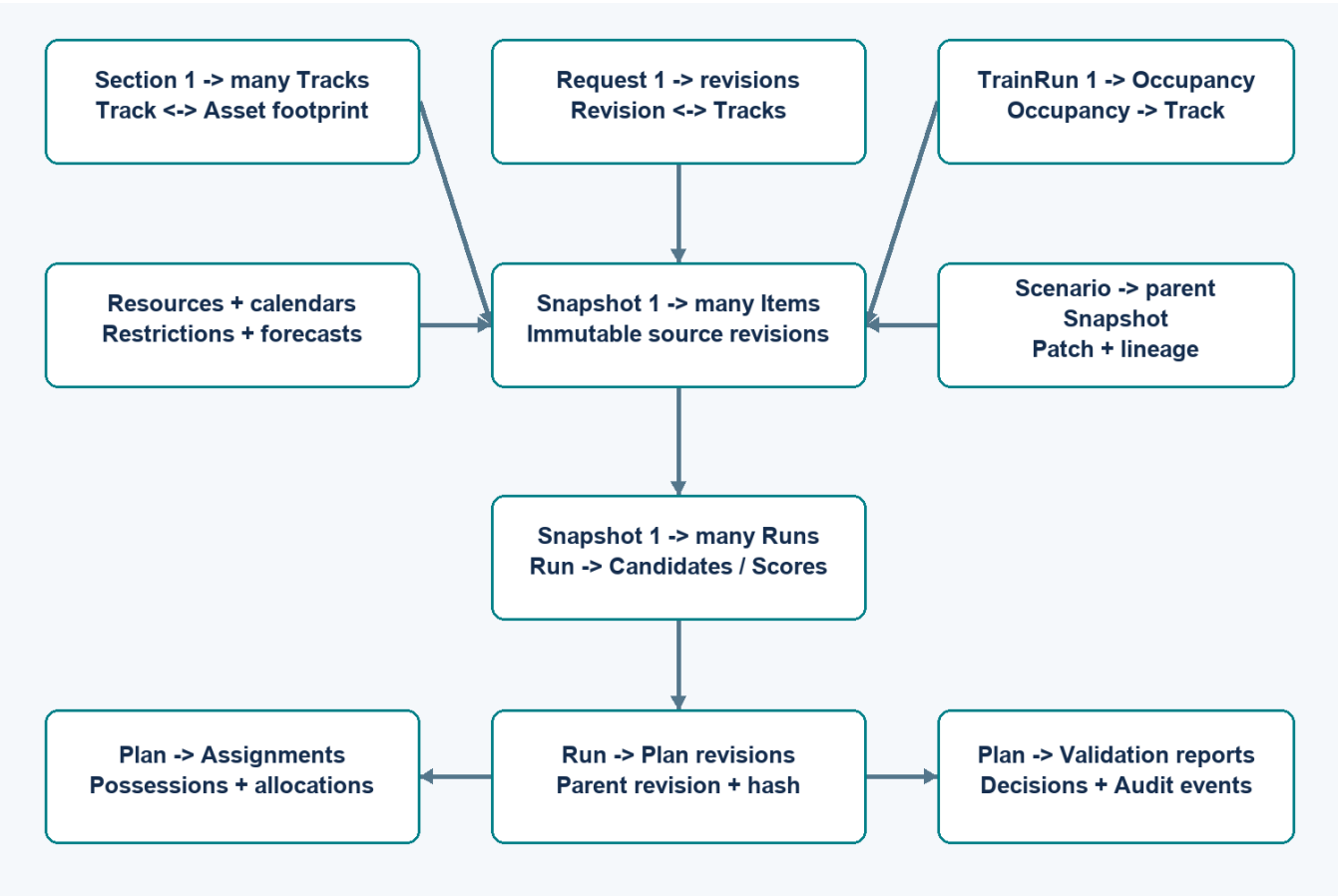
KPIs are calculated from persisted assignments and raw intervals. Display snapshot, horizon, units, planner status, mandatory-work feasibility and validator result beside every comparison. Never compare different workloads or count planned work as completed work.

Metric	Definition	Interpretation
Requests scheduled	Distinct selected request IDs	Planned coverage, not actual completion
Mandatory coverage	Mandatory requests scheduled / mandatory requests due	Undefined if none due; show numerator/denominator
On-time coverage	Selected requests restored/completed by their configured deadline / requests due	Show denominator including deferred work
Possession count	Distinct selected possession IDs	Separate by footprint where useful
Reserved track-minutes	Sum across tracks of union duration of reserved intervals	Avoid double counting overlap on a track
Planned track availability	$1 - \text{union}(\text{unavailable track intervals}) / \text{total in-scope track-horizon minutes}$	A capacity proxy; not measured asset reliability
Temporal block utilization	Union of active work time within each possession / possession duration, aggregated by duration	Parallel task-minutes are not summed beyond 100%
Forecast exposure	Declared expected movement-overlap proxy or scenario-specific overlap	Not predicted delay minutes unless a validated delay model exists
Plan churn	Changed previously approved future assignments / comparable previous assignments	Define moved/removed and exclude completed work
Validation failures	Counts by check code, for all evaluated plans	Includes zero only when actually checked
Runtime/quality	Wall time, objective, bound and gap	State candidate-search limitations

Actual asset availability requires observed outage and service-restoration records over an asset-specific denominator. Do not infer it from fewer possessions. Train delay improvement is not meaningful in the initial fixed-timetable model, where confirmed train overlaps must be zero for both valid planners.

For a lower-is-better metric, $\text{improvement} = (\text{baseline} - \text{RailSync}) / \text{baseline} * 100$ only when baseline is nonzero and outcomes are comparable. If zero, show absolute difference and percentage N/A. Report negative improvements. Run multiple fixed-seed normal, congested, critical-work, freight and resource-shortage scenarios; publish per-scenario results, medians/ranges and feasibility rates without selecting only favorable cases.

25. Database schema and integrity rules



Entity relationships — RailSync proposed design

Use UUID primary keys unless a composite key is explicitly stated. Use timestamptz for absolute time, integer minutes for durations, NUMERIC for bounded scores, JSONB for versioned payloads and text enums/check constraints for states. Persist created_at, actor and revision on mutable aggregates. PostgreSQL range types and overlap indexing support interval queries. [T4]

Table	Main columns and relationships
users / role_memberships	id; subject; department; active; role membership
stations / sections / tracks	station IDs; section from/to FKs; track section FK, direction, status
assets / asset_footprints	asset department/type; asset-to-track and optional km interval
isolation_zones / zone_tracks	electrical zone; affected track mapping and revision
maintenance_requests	id, ticket unique, department, current_revision, lifecycle_state
request_revisions	request FK, revision unique per request, severity, urgency, work_minutes, earliest_at, deadline_at, deadline_kind, mandatory, payload_hash
request_footprints / requirements	revision FK; track FK; skill/equipment type, quantity, setup/restore needs
import_batches / source_records	adapter, schema, received_at; external_id, source_revision, hash, raw payload, errors
trains / train_runs / occupancy	service; dated run; run+sequence, track, enter/exit, provenance
forecasts / coa_windows	issued_at, valid interval, footprint, expected_count, uncertainty semantics, source
restrictions / existing_blocks	type, interval, footprint, rule revision, operational status
resource_units / calendars	unit type, qualifications, home location; available interval, capacity

Table	Main columns and relationships
snapshots / snapshot_items	horizon, hash unique, source watermarks; immutable referenced revisions
planning_runs	snapshot FK, planner, configuration hashes, status, solver metadata
assessments / candidates	run FK; request score provenance; possession/task times, footprint, allocations, costs
plan_revisions / assignments	run FK, parent revision, hash, state; request revision, candidate, work interval
possessions / allocations	plan FK, footprint and interval; named/pool resource usage interval
validation_reports / checks	plan hash, snapshot hash, version, status; code and evidence payload
decisions / audit_events	plan FK, actor, action, reason, expected version; before/after hashes
scenarios / disruptions / kpi_results	parent snapshot and patch; event lineage; metric formula version and inputs

Use join tables for many-to-many footprints; do not store comma-separated track IDs. Index request state/deadline, occupancy(track_id, enter_at), run status and source identities. Range overlap indexes supplement, not replace, final transaction checks.

26. Database constraints and representative payloads

26.1 Required integrity controls

Enforce $\text{end_at} > \text{start_at}$, $\text{work_minutes} > 0$, score in $[0, 100]$, nonnegative capacity and quantities, and $\text{earliest_at} \leq \text{deadline_at}$ where a deadline exists. $\text{Unique}(\text{source_system}, \text{external_id}, \text{source_revision})$ prevents duplicate ingestion. $\text{Unique}(\text{plan_revision_id}, \text{request_id})$ prevents duplicate task assignment in one plan.

Scope active reservation overlap enforcement to approved operational plans, not competing draft or simulation plans. Named-resource reservations use exclusion/transactional overlap checks; pooled capacity uses locked calendar intervals plus recomputation. Approval atomically writes reservations and the decision. Serialize approvals within a corridor or acquire a consistent lock order across all affected resources.

26.2 Request example (synthetic)

```
{
  "department": "ENGINEERING",
  "asset_id": "fixture-asset-01",
  "footprint": ["fixture-track-AB"],
  "issue_type": "TRACK_INSPECTION",
  "severity": 3, "urgency": 4,
  "work_minutes": 60,
  "setup_minutes": 10, "restore_minutes": 10,
  "earliest_at": "2026-09-21T00:00:00+05:30",
  "deadline_at": "2026-09-22T06:00:00+05:30",
  "deadline_kind": "RESTORED_BY",
  "mandatory": false,
  "block_required": true, "power_block_required": false,
  "requirements": [{"type": "TRACK_CREW", "quantity": 1}],
  "source_mode": "SIMULATED"
}
```

Public production IDs are UUIDs; readable fixture IDs above are illustrative. Mandatory status changes require an authorized policy command and audit reason. Imported severity scales are mapped explicitly to the canonical scale before rule scoring.

26.3 Plan response contract

Return `run_id`, `snapshot_id`, `snapshot_hash`, `planner_type`, `solver_status`, `objective_terms`, `best_bound`, `gap`, `selected_possessions`, `assignments`, `deferred_requests`, `validation_report_id` and `review_state`. A null solution is distinct from an empty feasible selection. UI clients must not substitute mock assignments when either occurs.

27. API specification

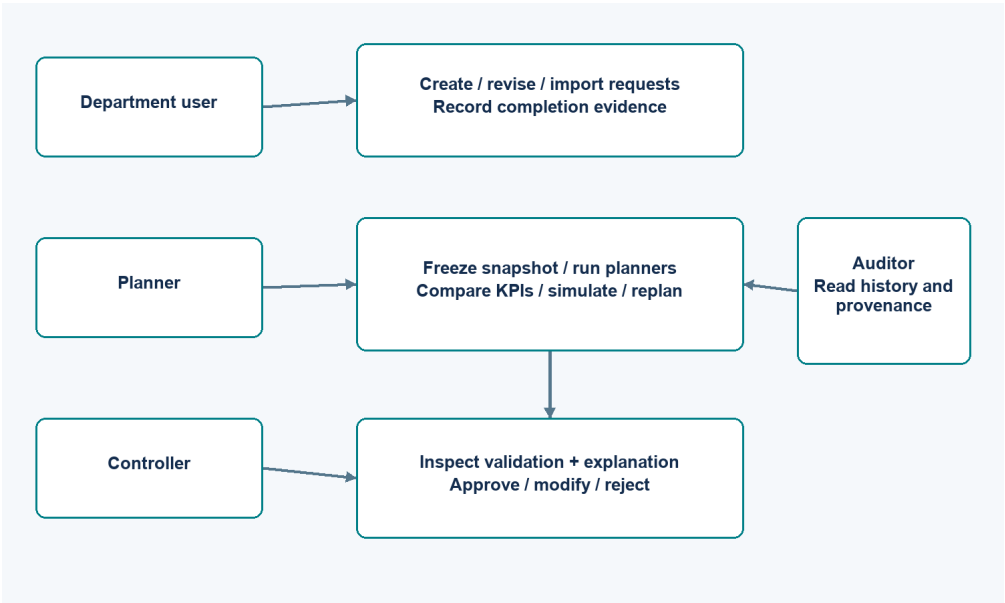
All routes are under `/api/v1`. Use OpenAPI generated from typed request/response models. Pagination uses cursor and limit; list endpoints support department, state, section and horizon filters. Times are ISO 8601 with explicit offset. Reject naive timestamps.

Method and route	Request / response	Permission
POST <code>/maintenance-requests</code>	Canonical request -> 201 id/revision	Department
GET <code>/maintenance-requests/{id}</code>	Request, revisions and planning membership	Scoped read
PATCH <code>/maintenance-requests/{id}</code>	Fields + If-Match -> new revision	Owning department
POST <code>/maintenance-requests/{id}/transitions</code>	action, reason, expected_version	Role by transition
POST <code>/imports/preview</code>	source, mapping, file -> batch/errors	Authorized importer
POST <code>/imports/{id}/commit</code>	expected preview hash -> counts	Authorized importer
GET <code>/network, /assets, /occupancy</code>	Horizon/scope -> canonical facts	Planner/read
POST <code>/snapshots</code>	horizon, scenario, source cutoffs -> manifest	Planner
POST <code>/planning-runs</code>	snapshot_id, BASELINE or CPSAT, config -> 202 job	Planner
GET <code>/planning-runs/{id}</code>	status, diagnostics and result links	Scoped read
POST <code>/planning-runs/{id}/cancel</code>	expected_version -> cancellation	Planner
GET <code>/plans/{id}</code>	exact revision, assignments and reasons	Scoped read
POST <code>/plans/{id}/validate</code>	revision hash -> 202 report job	Planner/controller
POST <code>/plans/{id}/modify</code>	pins/exclusions/resource changes -> child revision	Planner/controller
POST <code>/plans/{id}/approve</code>	expected hashes/revision, reason -> decision	Controller
POST <code>/plans/{id}/reject</code>	reason, expected_version -> decision	Controller
POST <code>/plans/{id}/replan</code>	event IDs, freeze policy -> new job	Planner/controller
POST <code>/scenarios</code>	parent snapshot + patch -> scenario ID	Planner
GET <code>/comparisons</code>	baseline_run, optimized_run -> checked KPI set	Scoped read
GET <code>/audit-events</code>	entity/time cursor -> evidence	Auditor
GET <code>/plans/{id}/export</code>	format, revision -> labeled artifact	Scoped read

Use 400 for malformed command, 401/403 for identity/permission, 404 for unavailable resource, 409 for stale version or conflicting approval, 422 for field/domain validation, and 503 for unavailable dependency. Error body: code, message, field_errors, evidence_ids, correlation_id. Never return internal secrets or stack traces.

Mutating imports, run creation and decisions accept Idempotency-Key bound to actor and request hash. Reuse with different content returns 409. A durable database job record exists before a 202 response. Polling or server events show progress; no long solver execution inside an API request.

28. Frontend screens and use cases



Usecase — RailSync proposed design

Build minimal forms and backend inspection first. The finished frontend reads real API responses exclusively and includes the following screens.

Screen	Main content	Required behavior
Department workspace	Requests, validation errors, create/revise form	Department scope, clear lifecycle, no final block-time field
Data readiness	Import batches, coverage, source freshness	Surface blocking gaps and simulation labels
Corridor view	Stations/tracks, occupancy and restrictions	Track/direction clarity; map is supplementary
Planning workspace	Horizon, snapshot, method, weights	Disable run when required facts are missing
Timeline	Trains, possessions, work intervals and resources	Distinct confirmed/proposed/approved states
Request explanation	Priority method, factors, candidate reasons	Separate prediction from scheduling logic
Controller review	Validator report, plan diff, approve/modify/reject	Failed/stale plans cannot be approved
Weekly/monthly board	Calendar, commitments and tentative work	Same plan data; explicit uncertainty
Baseline comparison	KPI definitions, values, deltas, run metadata	N/A, zero and negative gains displayed honestly
Scenario/replan	Changes, frozen work and alternative outcomes	Simulation isolation and infeasibility visible
Audit explorer	Revisions, decisions and evidence	Read-only history and downloadable evidence

Use accessible labels, keyboard navigation, readable contrast and text/icons alongside colors. Every chart has units, legend, time zone and an accessible table. Empty, loading, failed, stale, unknown and infeasible states are designed explicitly. Do not show invented chart values during loading or offline operation.

29. Technology stack and deployment

Layer	Selected technology	Why / implementation boundary
Web	Next.js, TypeScript, Tailwind CSS	Typed interactive screens and reusable styling; keep planning rules server-side [T6]
Charts/map	Recharts; optional Leaflet	Timeline/KPI visualization and corridor orientation; no dependency on a map for planning
API	Python, FastAPI, Pydantic	One language for rules/ML/optimization; typed validation and API contracts [T3, T7]
Persistence	PostgreSQL, SQLAlchemy, Alembic	Relational integrity, time intervals, transactions and migrations [T4]
Planning	OR-Tools CP-SAT	Integer/Boolean constraints and scheduling support [T1, T2]
ML	scikit-learn, XGBoost, SHAP	Rule comparison, tree models and interpretable feature attribution [R5, R6, T5]
Jobs	Separate Python worker with PostgreSQL job queue initially	Durable jobs without unnecessary distributed infrastructure
Tests	pytest, Hypothesis, API tests; Playwright for completed UI	Boundary/property tests and end-to-end decision workflow
Packaging	Docker Compose and locked Python/JS dependencies	Reproducible local/demo environment

Pin mutually compatible supported versions during M1 and record them in lockfiles. This document selects components, not untested version combinations. Prototype does not need microservices, Kafka, Spark or a separate model-serving cluster.

Deployment topology: browser -> TLS reverse proxy -> web/API; API -> PostgreSQL; worker -> PostgreSQL and restricted artifact storage. The database and worker are private. Worker CPU/memory and solver time are bounded. Secrets come from environment or deployment secret storage and are absent from source control.

Provide health/readiness endpoints, structured logs with correlation IDs, backups and a tested restore procedure. Recover jobs using leases/heartbeats so a crashed worker does not leave SOLVING forever. Re-running a recovered job must not duplicate approvals or reservations. A local demo can run on a declared 4-core/16-GB test machine; performance targets below are provisional and require measurement.

30. Security, auditability and nonfunctional requirements

ID	Shall requirement	Verification
NFR-01	Authorize every read/write by role and department/corridor scope	API negative tests for cross-scope access
NFR-02	Preserve immutable snapshots, revisions and decision history	Update/delete denial plus reconstruction test
NFR-03	Reject stale/conflicting approvals atomically	Concurrent approval test
NFR-04	Record source, model, code and configuration provenance	Export audit completeness assertion
NFR-05	Separate simulations from operational reservations	Scenario isolation test
NFR-06	Keep imports bounded and sanitized	File size/type limits, malicious CSV/text fixtures
NFR-07	Recover durable planning jobs after worker interruption	Kill/restart integration test
NFR-08	Preserve undefined metrics and computation failures	Null/zero/error API and UI tests
NFR-09	Provide accessible critical workflows	Keyboard and screen-reader labeling checks
NFR-10	Meet measured service targets on declared fixtures	Load/benchmark report with environment

Provisional performance targets: CRUD p95 under 500 ms for 20 concurrent demo users; snapshot/window generation under 5 seconds on the declared weekly fixture; solver budget 30 seconds for weekly and 120 seconds for monthly planning, returning truthful status when exceeded. These are acceptance targets to test, not measured results. Limit the first benchmark to 300 requests and a reported candidate cap; record input counts, hardware and dependency versions.

Use hashed passwords or an identity provider, secure sessions, CSRF protection when using cookies, rate limits and strict upload validation. Avoid unnecessary personal data in crew records. Restrict raw source payload access. Escape spreadsheet formula prefixes on CSV exports. Sanitize descriptions before rendering.

Audit events include actor, timestamp, action, entity/revision, correlation ID, reason, and before/after hashes. Application roles cannot edit audit history. A chained hash can expose accidental tampering but is not tamper-proof without external signed anchoring and protected storage. Retention periods and backup policy require project-owner decisions before operational deployment.

31. Functional SRS and traceability

“Shall” statements below are testable requirements. Priority is implementation urgency, not railway work priority. Section references define detailed behavior.

ID	Requirement	Priority / milestone	Acceptance test
FR-01	System shall create and validate department maintenance requests	P0 / M2	AT-01
FR-02	System shall import versioned source records idempotently	P0 / M2	AT-02
FR-03	System shall model explicit network/asset footprints	P0 / M3	AT-03
FR-04	System shall build dated occupancy and freight/COA inputs	P0 / M4	AT-04
FR-05	System shall freeze complete, reproducible planning snapshots	P0 / M5	AT-05
FR-06	System shall produce deterministic first-feasible baseline plans	P0 / M6	AT-06
FR-07	System shall provide traceable rule priority with gated ML experiments	P1 / M7	AT-07
FR-08	System shall derive conservative footprint availability	P1 / M8	AT-08
FR-09	System shall generate timed opportunities and exclusion reasons	P1 / M9	AT-09
FR-10	System shall enforce work compatibility and resource allocation	P1 / M10	AT-10
FR-11	System shall solve the declared CP-SAT model and expose true status	P1 / M11	AT-11
FR-12	System shall independently validate and fail closed	P1 / M12	AT-12
FR-13	System shall explain decisions and gate controller commands	P2 / M13	AT-13
FR-14	System shall produce consistent weekly/monthly schedules	P2 / M14	AT-14
FR-15	System shall compare both planners on identical facts	P2 / M15	AT-15
FR-16	System shall replan with frozen commitments and new approval	P3 / M16	AT-16
FR-17	System shall isolate and compare what-if scenarios	P3 / M17	AT-17
FR-18	System shall present real results through complete frontend flows	P3 / M18	AT-18
FR-19	System may add forecasting only after held-out baseline evaluation	P3 / M19	AT-19

Changes to these requirements require a versioned decision record describing reason, affected tests and migration impact. A milestone marked complete must point to backend output artifacts and passing automated tests. Mock UI demonstrations do not satisfy any planner acceptance test.

32. Automated testing and acceptance criteria

Test	Fixture/action	Required result
AT-01	Invalid duration, wrong asset department, concurrent request edit	422/403/409 as applicable; valid request persists
AT-02	Import same revision twice; include one malformed row	One canonical revision; precise quarantine report
AT-03	Double track and cross-section isolation	Correct footprint; unknown isolation blocks candidates
AT-04	Overnight dated run and unconfirmed freight envelope	Correct chronological occupancy; conservative reservation
AT-05	Mutate live source after snapshot	Original hash and contents unchanged
AT-06	Tiny known first-feasible fixture	Expected deterministic assignments and deferrals
AT-07	Fixed features and missing trained model	Recomputable rule contributions; RULE label
AT-08	Buffered 80-minute gap, 80/81-minute packages	Exact fit admitted; oversized package excluded
AT-09	No window, no resources, capped search	Distinct evidence-backed reason codes
AT-10	Overlapping crews, forbidden bundle and group hazard	All excluded by actual rules
AT-11	Enumerate all subsets of ≤ 12 candidates	Solver optimum equals brute-force optimum; infeasible case truthful
AT-12	Corrupt otherwise valid plan in each constraint family	FAIL/ERROR, never approvable
AT-13	Two conflicting approvals and stale hash	At most one commits; stale request returns 409
AT-14	Month boundary with carryover resource rest	No duplicated task or cross-boundary violation
AT-15	Same-snapshot comparison and mismatched comparison	Correct arithmetic; mismatch rejected
AT-16	Delay enters frozen possession	Stale flag and attention; no silent movement
AT-17	Scenario outage and invalid train patch	Parent unchanged; invalid patch rejected
AT-18	Browser create -> plan -> validate -> approve -> export	UI values equal persisted backend values
AT-19	Optional forecast evaluation	Temporal holdout, simple baseline and no unsupported quality claim

Use unit tests for scoring/intervals, property tests for interval union and conservative rounding, integration tests against real PostgreSQL, solver-oracle tests for tiny instances and mutation tests for validator independence. UI mocks may support isolated component tests but must never power the demonstration or acceptance run.

The final release gate requires all mandatory FR/NFR tests through M18, zero unresolved critical defects, reproducible installation, documented benchmark results, a successful backup restore and an evidence bundle linking commit, snapshot, configurations, plans and reports. M19 is optional and cannot delay the core release. No universal positive KPI improvement is required; correctness and truthful evaluation are required.

33. Numbered implementation roadmap — P0 foundation

Universal gate: milestone N+1 must not start until milestone N has automated tests passing and correct backend output checked against explicit expected results. Create a milestone evidence file containing scope, commit, commands, results, output paths and unresolved issues. “Page looks right” is not evidence. Resolve failures before advancing.

Milestone	Build exactly this	Dependencies	Gate / correct backend output
M1 · P0	Repo, backend skeleton, PostgreSQL, migrations, config, auth skeleton, test harness, dependency locks	None	Clean DB migrate + restart; health response; authorization smoke test; CI passes
M2 · P0	MaintenanceRequest revisions, department validation, lifecycle commands, CSV/JSON simulated/import adapters	M1	Persist/read valid request; AT-01/02; duplicate import creates no extra demand
M3 · P0	Stations, sections, tracks, assets, footprints, isolation mapping and restrictions	M2	Referentially valid corridor fixture; AT-03
M4 · P0	TrainRun, dated timetable, section occupancy, COA source contract and freight envelopes	M3	Ordered overnight occupancy and known coverage gaps; AT-04
M5 · P0	Planning snapshots, immutable manifest, hashes, source cutoffs and completeness checks	M4	Reconstruct unchanged input after source mutation; AT-05
M6 · P0	First-feasible single-request baseline; minimal interval/resource feasibility and stored results	M5	Exact fixture schedule and reason codes; AT-06; no approval feature enabled

Do not start M2 with a trained priority model. Request schema and basic department authorization exist first. At M3, replace any provisional request location string with validated footprint references through an explicit migration. At M4, simulated records must obey physical route consistency even though their values are synthetic.

M6 necessarily includes basic window intersection and resource checks so the baseline can work. This is a small dependency slice, not permission to build the complete P1 optimizer early. Later P1 improvements must keep the baseline's feasibility semantics aligned for fair comparison.

P0 completion means a developer can submit real input through an API, freeze it, run the baseline, restart the service and retrieve exactly the persisted output. It does not mean an approved operational plan exists.

34. Numbered implementation roadmap — P1 core intelligence

Milestone	Build exactly this	Dependencies	Gate / correct backend output
M7 · P1	Rule priority formula, contributions, feature versions; RF/XGBoost experiment interface	M6	AT-07; exact score arithmetic; absent data/model gives disclosed fallback
M8 · P1	Full Corridor Availability Engine with COA intersection, freight policy and footprint constraints	M7, M4–M5	AT-08; interval golden fixtures and property tests pass
M9 · P1	Maintenance Opportunity Engine, explicit times, costs, exclusion codes, search caps	M8	AT-09; candidates all respect work/setup/restore semantics
M10 · P1	Compatibility matrix, group rules, bundles, resource allocation, travel/rest	M9, M3	AT-10; recomputable task subintervals and concrete resource use
M11 · P1	CP-SAT candidate selection, hard constraints, configurable objective, solver metadata	M10	AT-11; tiny-instance brute-force oracle; truthful timeout/infeasible output
M12 · P1	Independent validator and mutation suite; validate both baseline and optimized plans	M11, M5	AT-12; no failed/error report can become reviewable

ML training is conditional inside M7: if valid labels are unavailable, deliver the experiment pipeline and model-card template while keeping the rule baseline active. Lack of defensible labels does not justify fabricated scores and does not prevent deterministic core progress. The gate applies to the implemented backend behavior, not an unsupported claim that ML has already succeeded.

At M11, model construction is independently inspectable through a candidate manifest and constraint diagnostics. Store actual selected candidates; never return a canned successful plan. If candidate count grows excessively, reduce the scenario or introduce documented pruning and rerun the acceptance suite.

P1 completion means feasible outputs are independently checked and invalid outputs are blocked. It still does not mean controller workflow, weekly/monthly reporting or the final frontend are finished.

35. Numbered implementation roadmap — P2 and P3

Milestone	Build exactly this	Dependencies	Gate / correct backend output
M13 · P2	Structured explanations; approve/modify/reject; optimistic locking and audit	M12	AT-13; validation hash and operational revision bound to approval
M14 · P2	Weekly/monthly orchestration, firm/tentative status and exports	M13	AT-14; same model and boundary correctness
M15 · P2	Baseline comparison service, metric formulas, evidence export	M14, M6	AT-15; hand-checked KPI fixture and mismatch rejection
M16 · P3	Event-driven rolling-horizon replan, freeze policy, churn, stale-run protection	M15	AT-16; old unsafe plan flagged and replacement separately validated
M17 · P3	Scenario patches, what-if planning and lineage comparisons	M16	AT-17; scenario cannot write active reservations
M18 · P3	Complete frontend, accessibility, deployment scripts, benchmark and demo evidence	M17	AT-18; full real-backend journey; release gate in §32
M19 · P3 optional	Forecasting experiment; ANN only if justified against simple baselines	M18 and valid historical data	AT-19; no promotion without agreed evaluation thresholds

Minimal input/review screens may accompany each completed backend milestone. Production-quality presentation belongs to M18; frontend polish must not consume the effort required to make earlier backend outputs correct.

The strict sequential gate applies across these milestones, including P3. If a feature is intentionally deferred, update the scope contract and acceptance matrix explicitly before calling a reduced release complete. Do not relabel a half-implemented milestone as passed.

For each milestone, Codex must report: what changed, which requirements it satisfies, exact test commands and results, sample persisted output, remaining limitations, and the next milestone. No completion claim may rely on unrun tests. If blocked by missing domain rules, use a labeled synthetic fixture for development and keep operational approval disabled for unknown rules.

36. Demonstration scenario and expected evidence

36.1 Small hand-checkable scenario

Use stations A–B–C–D with one track per section. On A–B, confirmed traffic plus five-minute margins yields a single 01:55–03:15 window. Requests ENG-01 (60 work minutes) and TRD-01 (40 work minutes) share the A–B footprint. Fixture policy allows parallel work with ten minutes shared setup and ten minutes restoration, using distinct eligible crews and valid isolation. Their combined possession takes 80 minutes. These permissions are synthetic test assumptions.

SNT-01 requires 90 work minutes plus setup/restoration and cannot fit that window. It should have a WINDOW_TOO_SHORT exclusion unless a later valid opportunity exists. A fourth request on B–C competes for one of the same crews, exercising resource/travel logic. Make an alternate scenario with forbidden ENG/TRD compatibility; the bundle must disappear.

This fixture demonstrates why sharing setup and concurrent compatible work can help. The actual baseline and CP-SAT outputs must be computed at runtime. Do not present these described expectations as measured demo results before the software exists.

36.2 Judge walkthrough

1. Show SIMULATED data provenance, source coverage and department requests.
2. Open the occupancy timeline and inspect the derived window boundaries.
3. Freeze a snapshot and run the baseline; show saved assignments and deferrals.
4. Show rule priority with exact contributions, then run CP-SAT.
5. Inspect actual bundle task intervals, resource assignment, solver status and objective terms.
6. Run independent validation; deliberately mutate a separate test plan to show rejection.
7. Compare real KPIs for the same snapshot, including any unchanged or worse metric.
8. Approve the valid revision as controller, then inject a train delay in a scenario.
9. Show staleness/frozen-work behavior and the new proposal or truthful infeasibility.
10. Export the plan, validation report and provenance bundle.

Prepare normal, high-freight, resource-outage, urgent-defect and infeasible fixtures with fixed seeds. A network-free demonstration uses local synthetic data, not fake API responses. Recovery from a worker restart is part of rehearsal.

37. Risk register and mitigation

Risk	Consequence	Mitigation / owner
Official PS text changes or differs	Misaligned submission	Team lead archives organizer-issued statement and updates traceability
Real system fields unavailable	Incorrect adapter claims	Data lead documents project-defined mappings and simulation labels
Unknown operational rules	Unsafe-looking proposal	Domain reviewer supplies policy; validator blocks unknowns
Poor or biased ML labels	Misleading risk estimates	ML owner keeps rule fallback and documents label/intervention bias
Excessive candidate combinations	Slow or incomplete search	Optimizer owner logs caps, benchmarks and labels restricted optimality
Optimizer/validator share a bug	Invalid plan passes	Validation owner implements separate checks and mutation tests
Stale data or concurrent approval	Conflicting commitments	Backend owner uses revisions, locks, freshness checks and atomic reservations
KPI gaming	False success claims	Evaluation owner fixes denominators and compares identical inputs
Ambiguous possession versus work time	Underestimated duration	Domain model stores setup, work, restoration and footprint separately
Simulations alter live state	Unauthorized reservation	Backend owner enforces scenario isolation and explicit promotion workflow
Frontend hides computation errors	Fake confidence	UI owner designs unknown, stale, infeasible and failure states

The prototype is a decision-support demonstration. Field deployment needs validated railway integration contracts, complete operating rules, human procedures and independent operational assurance. This boundary follows directly from the modeled-system limits; a software PASS report must be described as compliance with configured prototype constraints.

Resolve acceptance-affecting risks in the milestone evidence. Do not hide them in presentation notes. A known limitation can remain only if the requirement is explicitly scoped and the UI communicates the boundary.

38. References and evidence register

All links checked or retrieved on 15 September 2026. Source labels are cited near the claims they support. Technical specifications not attributed to a paper are proposed RailSync engineering decisions. Retrieval timestamps shown by search engines are not publication dates.

S1. SIH 2026 community archive, PS SIH26027. Matching reproduced PS text; explicitly unofficial.

<https://sih2026.vuce.in/ps/SIH26027>

S2. SIH 2026 Explorer, PS SIH26027. Secondary mirror used to cross-check the required integration and planning horizons. <https://sih-explorer.amanuniyal47.workers.dev/problem/SIH26027>

S3. Smart India Hackathon official problem-statement portal. Official source to confirm before submission; retrieval unsuccessful during this preparation. <https://sih.gov.in/sih2026PS>

R1. Tomas Lidén and Martin Joborn (2017). An optimization model for integrated planning of railway traffic and network maintenance. Transportation Research Part C, 74, 327–347. Author manuscript:

<https://liu.diva-portal.org/smash/get/diva2%3A1052249/FULLTEXT01.pdf>

R2. Resource considerations for integrated planning of railway traffic and maintenance windows (2018). University repository record; scope includes crew availability and work-time constraints.

<https://rune.une.edu.au/entities/publication/ee4602db-f694-41d0-9d52-2f3412e862c6>

R3. Development of a maintenance possession scheduler for a railway (2023). South African Journal of Industrial Engineering, 34(2). <https://doi.org/10.7166/34-2-2750>

R4. A heuristic approach to integrate train timetabling, platforming, and railway network maintenance scheduling decisions (2022). Transportation Research Part B, 158, 210–238.

<https://doi.org/10.1016/j.trb.2022.02.002>

R5. Tianqi Chen and Carlos Guestrin (2016). XGBoost: A Scalable Tree Boosting System.

<https://arxiv.org/abs/1603.02754>

R6. Scott M. Lundberg and Su-In Lee (2017). A Unified Approach to Interpreting Model Predictions.

<https://arxiv.org/abs/1705.07874>

T1. Google OR-Tools. CP-SAT Solver: integer modeling and result statuses.

https://developers.google.com/optimization/cp/cp_solver

T2. Google OR-Tools. Scheduling documentation: interval variables and scheduling constraints.

<https://github.com/google/or-tools/blob/stable/ortools/sat/docs/scheduling.md>

T3. FastAPI official documentation. <https://fastapi.tiangolo.com/>

T4. PostgreSQL official documentation: range types. <https://www.postgresql.org/docs/current/rangetypes.html>

T5. scikit-learn official documentation: ensemble methods.

<https://scikit-learn.org/stable/modules/ensemble.html>

T6. Next.js official documentation. <https://nextjs.org/docs>

T7. Pydantic official documentation: validation. <https://docs.pydantic.dev/latest/>

No RailSync runtime, accuracy, delay reduction or availability gain is asserted by these references. Paper findings motivate design choices; project results require the evaluation procedure in §24 and §32.

39. Codex Build Contract / Master Implementation Prompt

You are implementing RailSync AI, a SIH 2026 PS26027 railway fixed-infrastructure maintenance decision-support prototype. Treat this document as the implementation baseline. Build working backend behavior and automated evidence before presentation. Do not implement railway train control or claim access to live railway systems.

Follow this exact architecture: Engineering/TRD/S&T maintenance requests and imported/simulated TMS/SMMS/TDMS records + network/assets + timetable/dated train-section occupancy + COA/corridor inputs + goods forecasts + restrictions/resources/commitments -> immutable Integrated Railway Planning State -> rule/ML priority and Corridor Availability Engine -> Maintenance Opportunity Engine -> compatibility/bundling/resource allocation -> OR-Tools CP-SAT -> independent fail-closed validator -> structured explanations and controller approve/modify/reject -> execution records and later replanning. Run a fair first-feasible baseline on the same facts.

Use a modular Python/FastAPI/Pydantic backend, PostgreSQL with migrations, a durable worker process, OR-Tools CP-SAT, and a Next.js/TypeScript frontend. Use Random Forest/XGBoost/SHAP only through an evaluated, versioned ML pipeline with honest rule fallback. An ANN is optional future forecasting only. Lock compatible dependencies and preserve code/configuration/model/source versions.

Build in this numbered order, with no skipping:

1. M1 P0: repository, database, migrations, configuration, auth skeleton and automated test harness.
2. M2 P0: MaintenanceRequest system, department validation, revisions, lifecycle and idempotent import adapters.
3. M3 P0: network, tracks, assets, footprints, electrical zones and restrictions.
4. M4 P0: timetable, dated runs, section occupancy, COA input and freight uncertainty.
5. M5 P0: immutable planning snapshots and provenance.
6. M6 P0: deterministic first-feasible baseline with basic interval/resource correctness.
7. M7 P1: rule priority and gated model experiment interface.
8. M8 P1: complete Corridor Availability Engine.
9. M9 P1: timed opportunity generation and evidence-backed exclusions.
10. M10 P1: compatibility, bundle timing and resource allocation.
11. M11 P1: genuine CP-SAT model, objectives, solver statuses and diagnostics.
12. M12 P1: independent validator with deliberate corruption tests.
13. M13 P2: explanations, controller decisions, concurrency and audit.
14. M14 P2: consistent weekly/monthly planning and exports.
15. M15 P2: baseline-versus-RailSync KPI evaluation on identical inputs.
16. M16 P3: rolling-horizon dynamic replanning and frozen work.
17. M17 P3: isolated what-if scenarios.
18. M18 P3: complete frontend, deployment, benchmarks and demonstration evidence.

19. M19 P3 optional: forecasting experiments only if suitable data exists.

Milestone N+1 MUST NOT start until milestone N has passing automated tests AND correct backend output checked against explicit expected results. Maintain a milestone evidence record with commit, test commands/results and persisted output examples. Correct failures before advancing. Do not describe unrun tests as passing. Deliver the full authorized core through M18; M19 is optional.

Prohibit fake/hardcoded optimizer outputs, KPI improvements, conflicts and AI scores. Synthetic input fixtures are permitted only when labeled and processed by real backend code. Never make the baseline artificially bad. Never force positive improvements. Unknown, infeasible, empty, stale and error states must remain visible. No canned schedules or frontend fallback values may be presented as computed output.

Use explicit track footprints and half-open timezone-aware intervals. Confirmed occupancy and configured clearance, possession, electrical, resource, precedence and mandatory deadline constraints are hard. ML score and objective weights cannot override them. Initial train times are fixed. Preserve setup/work/restoration separately. Use conservative rounding and explicit freight uncertainty policy. Unknown critical facts block review/approval.

Implement the fixed-candidate CP-SAT model in §18 first. A candidate includes task subintervals and concrete resource assignments. Enforce at-most-once request coverage, mandatory coverage, candidate incompatibility, resource capacities, precedence and frozen work. Record pruning and qualify optimality to the generated candidate set. Return actual solver status, objective terms, best bound and runtime; never fabricate a solution after timeout.

Implement validator logic separately from candidate generation. Recompute constraints from raw snapshot facts. FAIL, ERROR, stale data and mismatched hashes block approval. Controller edits create new revisions and require revalidation. Approval atomically checks current operational revision and reserves resources; simulations never write operational reservations.

Compute KPIs from saved intervals with declared units and denominators. Distinguish planned track availability from actual asset reliability and forecast exposure from measured train delay. Compare the same snapshot and requirements, show negative gains and N/A percentages when baseline is zero. Planned tasks are not completed tasks.

Start now with M1, then proceed through the gates. Inspect the existing repository before editing, preserve unrelated user work, and update the requirement traceability as implementation progresses. When a domain rule is unavailable, document it, use only clearly labeled synthetic fixtures for development, and keep unknown operational cases fail-closed. At each milestone report what changed, test evidence, backend output, limitations and next step.